
Unifying Data with Sequences

Programs manipulate data. At the lowest level, programs work with structures such as strings, lists, vectors, maps, sets, and trees. At a higher level, these same data structure abstractions crop up again and again. For example:

- XML data is a tree.
- Database result sets can be viewed as lists or vectors.
- Directory hierarchies are trees.
- Files are often viewed as one big string or as a vector of lines.

In Clojure, all these data structures can be accessed through a single abstraction: the sequence (or *seq*).

A *seq* (pronounced “seek”) is a *logical* list. It’s logical because Clojure does not tie sequences to *implementation details* of a list such as a Lisp cons cell (see [The Origin of Cons, on page 58](#) for the history of cons). Instead, the *seq* is an abstraction that can be used everywhere.

Collections that can be viewed as *seqs* are called *seq-able* (pronounced “SEEK-a-bull”). In this chapter, you will meet a variety of *seq-able* collections:

- All Clojure collections
- All Java collections
- Java arrays and strings
- Regular expression matches
- Directory structures
- I/O streams
- XML trees

You will also meet the sequence library, a set of functions that can work with any *seq-able*. Because so many things are sequences, the sequence library is much more powerful and general than the collection APIs in most languages. The sequence library includes functions to create, filter, and transform data.

These functions act as the Collections API for Clojure, and they also replace many of the loops you would write in an imperative language.

In this chapter, you will become a power user of Clojure sequences. You will see how to use a common set of very expressive functions with an incredibly wide range of datatypes. Then, in the next chapter ([Chapter 4, *Functional Programming, on page 85*](#)), you will learn the functional style in which the sequence library is written.

3.1 Everything Is a Sequence

Every aggregate data structure in Clojure can be viewed as a sequence. A sequence has three core capabilities:

- You can get the first item in a sequence:

```
(first aseq)
```

first returns nil if its argument is empty or nil.

- You can get everything after the first item, in other words, the rest of a sequence:

```
(rest aseq)
```

rest returns an empty seq (not nil) if there are no more items.

- You can construct a new sequence by adding an item to the front of an existing sequence. This is called consing:

```
(cons elem aseq)
```

Under the hood, these three capabilities are declared in the Java interface `clojure.lang.ISeq`. (Keep this in mind when reading about Clojure, because the name `ISeq` is often used interchangeably with `seq`.)

The `seq` function will return a `seq` on any `seq`-able collection:

```
(seq coll)
```

`seq` will return nil if its `coll` is empty or nil. The `next` function will return the `seq` of items after the first:

```
(next aseq)
```

`(next aseq)` is equivalent to `(seq (rest aseq))`. [Table 4, *Clarifying rest/next behavior, on page 57*](#) clarifies the `rest/next` behavior.

If you have a Lisp background, you expect to find that the `seq` functions work for lists:

Form	Result
(rest ())	()
(next ())	nil
(seq (rest ()))	nil

Table 4—Clarifying rest/next behavior

```
(first '(1 2 3))
-> 1
```

```
(rest '(1 2 3))
-> (2 3)
```

```
(cons 0 '(1 2 3))
-> (0 1 2 3)
```

In Clojure, the same functions will work for other data structures as well. You can treat vectors as seqs:

```
(first [1 2 3])
-> 1
```

```
(rest [1 2 3])
-> (2 3)
```

```
(cons 0 [1 2 3])
-> (0 1 2 3)
```

When you apply rest or cons to a vector, the result is a seq, not a vector. In the REPL, seqs print just like lists, as you can see in the earlier output. You can check the actual returned type by taking its class:

```
(class (rest [1 2 3]))
-> clojure.lang.PersistentVector$ChunkedSeq
```

The \$ChunkedSeq at the end of the class name is Java's way of mangling nested class names. Seqs that you produce from a specific collection type are often implemented as a ChunkedSeq class nested inside the original collection class (PersistentVector in this example).

The generality of seqs is very powerful, but sometimes you want to produce a specific implementation type. This is covered in [Section 3.5, Calling Structure-Specific Functions, on page 76](#).

You can treat maps as seqs, if you think of a key/value pair as an item in the sequence:

The Origin of Cons

Clojure's sequence is an abstraction based on Lisp's concrete lists. In the original implementation of Lisp, the three fundamental list operations were named `car`, `cdr`, and `cons`. `car` and `cdr` are acronyms that refer to implementation details of Lisp on the original IBM 704 platform. Many Lisps, including Clojure, replace these esoteric names with the more meaningful names `first` and `rest`.

The third function, `cons`, is short for construct. Lisp programmers use `cons` as a noun, verb, and adjective. You use `cons` to create a data structure called a *cons cell*, or just a *cons* for short.

Most Lisps, including Clojure, retain the original `cons` name, since “construct” is a pretty good mnemonic for what `cons` does. It also helps remind you that sequences are immutable. For convenience, you might say that `cons` adds an element to a sequence, but it is more accurate to say that `cons` *constructs* a new sequence, which is like the original sequence but with one element added.

```
(first {:fname "Aaron" :lname "Bedra"})
-> [:lname "Bedra"]

(rest {:fname "Aaron" :lname "Bedra"})
-> ({:fname "Aaron"})

(cons [:mname "James"] {:fname "Aaron" :lname "Bedra"})
-> ([[:mname "James"] [:lname "Bedra"] [:fname "Aaron"]])
```

You can also treat sets as seqs:

```
(first #{:the :quick :brown :fox})
-> :brown

(rest #{:the :quick :brown :fox})
-> (:quick :fox :the)

(cons :jumped #{:the :quick :brown :fox})
-> (:jumped :brown :quick :fox :the)
```

Maps and sets have a stable traversal order, but that order depends on implementation details, and you should not rely on it. Elements of a set will not necessarily come back in the order that you put them in:

```
#{:the :quick :brown :fox}
-> #{:brown :quick :fox :the}
```

If you want a reliable order, you can use this:

```
(sorted-set & elements)
```

`sorted-set` will sort the values by their natural sort order:

```
(sorted-set :the :quick :brown :fox)
-> #{:brown :fox :quick :the}
```

Likewise, key/value pairs in maps won't necessarily come back in the order you put them in:

```
{:a 1 :b 2 :c 3}
-> {:a 1, :c 3, :b 2}
```

You can create a sorted map with `sorted-map`:

```
(sorted-map & elements)
```

sorted-maps won't come back in the order you put them in either, but they *will* come back sorted by key:

```
(sorted-map :c 3 :b 2 :a 1)
-> {:a 1, :b 2, :c 3}
```

In addition to the core capabilities of `seq`, two other capabilities are worth meeting immediately: `conj` and `into`.

```
(conj coll element & elements)
```

```
(into to-coll from-coll)
```

`conj` adds one or more elements to a collection, and `into` adds all the items in one collection to another. Both `conj` and `into` add items at an efficient insertion spot for the underlying data structure. For lists, `conj` and `into` add to the front:

```
(conj '(1 2 3) :a)
-> (:a 1 2 3)

(into '(1 2 3) '(:a :b :c))
-> (:c :b :a 1 2 3)
```

For vectors, `conj` and `into` add elements to the back:

```
(conj [1 2 3] :a)
-> [1 2 3 :a]

(into [1 2 3] [:a :b :c])
-> [1 2 3 :a :b :c]
```

Because `conj` (and related functions) do the efficient thing for the underlying data structure, you can often write code that is both efficient and completely decoupled from a specific underlying implementation.

The Clojure sequence library is particularly suited for large (or even infinite) sequences. Most Clojure sequences are *lazy*: they generate elements only

**Joe asks:****Why Do Functions on Vectors Return Lists?**

When you try examples at the REPL, the results of `rest` and `cons` appear to be lists, even when the inputs are vectors, maps, or sets. Does this mean that Clojure is converting everything to a list internally? No! The sequence functions always return a `seq`, regardless of their inputs. You can verify this by checking the Java type of the returned objects:

```
(class '(1 2 3))
-> clojure.lang.PersistentList

(class (rest [1 2 3]))
-> clojure.lang.PersistentVector$ChunkedSeq
```

As you can see, the result of `(rest [1 2 3])` is some kind of `Seq`, not a `List`. So, why does the result appear to be a list?

The answer lies in the REPL. When you ask the REPL to display a sequence, all it knows is that it has a sequence. It does not know what kind of collection the sequence was built from. So, the REPL prints all sequences the same way: it walks the entire sequence, printing it as a list.

when they are actually needed. Thus, Clojure's sequence functions can process sequences too large to fit in memory.

Clojure sequences are *immutable*: they never change. This makes it easier to reason about programs and means that Clojure sequences are safe for concurrent access. It does, however, create a small problem for human language. English-language descriptions flow much more smoothly when describing mutable things. Consider the following two descriptions for a hypothetical sequence function triple:

- triple triples each element of a sequence.
- triple takes a sequence and returns a new sequence with each element of the original sequence tripled.

The latter version is specific and accurate. The former is much easier to read, but it might lead to the mistaken impression that a sequence is actually changing. Don't be fooled: *sequences never change*. If you see the phrase "foo changes x," mentally substitute "foo returns a changed copy of x."

3.2 Using the Sequence Library

The Clojure sequence library provides a rich set of functionality that can work with any sequence. If you come from an object-oriented background where

nouns rule, the sequence library is truly “Revenge of the Verbs.”¹ The functions provide a rich backbone of functionality that can take advantage of any data structure that obeys the basic first/rest/cons contract.

The following functions are grouped into four broad categories:

- Functions that create sequences
- Functions that filter sequences
- Sequence predicates
- Functions that transform sequences

These divisions are somewhat arbitrary. Since sequences are immutable, *most* of the sequence functions create new sequences. Some of the sequence functions both filter and transform. Nevertheless, these divisions provide a rough road map through a large library.

Creating Sequences

In addition to the sequence literals, Clojure provides a number of functions that create sequences. `range` produces a sequence from a start to an end, incrementing by step each time.

```
(range start? end step?)
```

Ranges include their start but not their end. If you do not specify them, start defaults to zero, and step defaults to 1. Try creating some ranges at the REPL:

```
(range 10)
-> (0 1 2 3 4 5 6 7 8 9)

(range 10 20)
-> (10 11 12 13 14 15 16 17 18 19)

(range 1 25 2)
-> (1 3 5 7 9 11 13 15 17 19 21 23)
```

The `repeat` function repeats an element `x` `n` times:

```
(repeat n x)
```

Try to repeat some items from the REPL:

```
(repeat 5 1)
-> (1 1 1 1 1)

(repeat 10 "x")
-> ("x" "x" "x" "x" "x" "x" "x" "x" "x" "x")
```

1. Steve Yegge’s “Execution in the Kingdom of Nouns” (<http://tinyurl.com/the-kingdom-of-nouns>) argues that object-oriented programming has pushed nouns into an unrealistically dominant position and that it is time for a change.

Both range and repeat represent ideas that can be extended infinitely. You can think of iterate as the infinite extension of range:

```
(iterate f x)
```

iterate begins with a value x and continues forever, applying a function f to each value to calculate the next.

If you begin with 1 and iterate with inc, you can generate the whole numbers:

```
(take 10 (iterate inc 1))
-> (1 2 3 4 5 6 7 8 9 10)
```

Since the sequence is infinite, you need another new function to help you view the sequence from the REPL.

```
(take n sequence)
```

take returns a lazy sequence of the first n items from a collection and provides one way to create a finite view onto an infinite collection.

The whole numbers are a pretty useful sequence to have around, so let's defn them for future use:

```
(defn whole-numbers [] (iterate inc 1))
-> #'user/whole-numbers
```

When called with a single argument, repeat returns a lazy, infinite sequence:

```
(repeat x)
```

Try repeating some elements at the REPL. Don't forget to wrap the result in a take:

```
(take 20 (repeat 1))
-> (1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
```

The cycle function takes a collection and cycles it infinitely:

```
(cycle coll)
```

Try cycling some collections at the REPL:

```
(take 10 (cycle (range 3)))
-> (0 1 2 0 1 2 0 1 2 0)
```

The interleave function takes multiple collections and produces a new collection that interleaves values from each collection until one of the collections is exhausted.

```
(interleave & colls)
```

When one of the collections is exhausted, the `interleave` stops. So, you can mix finite and infinite collections:

```
(interleave (whole-numbers) ["A" "B" "C" "D" "E"])
-> (1 "A" 2 "B" 3 "C" 4 "D" 5 "E")
```

Closely related to `interleave` is `interpose`, which returns a sequence with each of the elements of the input collection segregated by a separator:

```
(interpose separator coll)
```

You can use `interpose` to build delimited strings:

```
(interpose "," ["apples" "bananas" "grapes"])
-> ("apples" ", " "bananas" ", " "grapes")
```

`interpose` works nicely with `(apply str ...)` to produce output strings:

```
(apply str (interpose \, ["apples" "bananas" "grapes"]))
-> "apples,bananas,grapes"
```

The `(apply str ...)` idiom is common enough that Clojure wraps it as `clojure.string/join`:

```
(join separator sequence)
```

Use `clojure.string/join` to comma-delimit a list of words:

```
(use '[clojure.string :only (join)])
(join \, ["apples" "bananas" "grapes"])
-> "apples,bananas,grapes"
```

For each collection type in Clojure, there is a function that takes an arbitrary number of arguments and creates a collection of that type:

```
(list & elements)
```

```
(vector & elements)
```

```
(hash-set & elements)
```

```
(hash-map key-1 val-1 ...)
```

`hash-set` has a cousin `set` that works a little differently: `set` expects a collection as its first argument:

```
(set [1 2 3])
-> #{1 2 3}
```

`hash-set` takes a variable list of arguments:

```
(hash-set 1 2 3)
-> #{1 2 3}
```

vector also has a cousin, `vec`, that takes a single collection argument instead of a variable argument list:

```
(vec (range 3))
-> [0 1 2]
```

Now that you have the basics of creating sequences, you can use other Clojure functions to filter and transform them.

Filtering Sequences

Clojure provides a number of functions that filter a sequence, returning a subsequence of the original sequence. The most basic of these is `filter`:

```
(filter pred coll)
```

`filter` takes a predicate and a collection and returns a sequence of objects for which the filter returns true (when interpreted in a boolean context). You can filter the whole-numbers from the previous section to get the odd numbers or the even numbers:

```
(take 10 (filter even? (whole-numbers)))
-> (2 4 6 8 10 12 14 16 18 20)
```

```
(take 10 (filter odd? (whole-numbers)))
-> (1 3 5 7 9 11 13 15 17 19)
```

You can take from a sequence while a predicate remains true with `take-while`:

```
(take-while pred coll)
```

For example, to take all the characters in a string up to the first vowel, use this:

```
(take-while (complement #{\a\e\i\o\u}) "the-quick-brown-fox")
-> (\t \h)
```

There are a couple of interesting things happening here:

- Sets also act as functions. So, you can read `#{\a\e\i\o\u}` either as “the set of vowels” or as “the function that tests to see whether its argument is vowel.”
- `complement` reverses the behavior of another function. The previous complemented function tests to see whether its argument is *not* a vowel.

The opposite of `take-while` is `drop-while`:

```
(drop-while pred coll)
```

`drop-while` drops elements from the beginning of a sequence while a predicate is true and then returns the rest. You could `drop-while` to drop all leading non-vowels from a string:

```
(drop-while (complement #{\a\e\i\o\u}) "the-quick-brown-fox")
-> (\e \- \q \u \i \c \k \- \b \r \o \w \n \- \f \o \x)
```

`split-at` and `split-with` will split a collection into two collections:

```
(split-at index coll)
```

```
(split-with pred coll)
```

`split-at` takes an index, and `split-with` takes a predicate:

```
(split-at 5 (range 10))
->[(0 1 2 3 4) (5 6 7 8 9)]
```

```
(split-with #(<= % 10) (range 0 20 2))
->[(0 2 4 6 8 10) (12 14 16 18)]
```

All the `take-`, `split-`, and `drop-` functions return lazy sequences, of course.

Sequence Predicates

Filter functions take a predicate and return a sequence. Closely related are the sequence predicates. A sequence predicate asks how some other predicate applies to every item in a sequence. For example, the `every?` predicate asks whether some other predicate is true for every element of a sequence.

```
(every? pred coll)
```

```
(every? odd? [1 3 5])
-> true
```

```
(every? odd? [1 3 5 8])
-> false
```

A lower bar is set by some:

```
(some pred coll)
```

`some` returns the first nonfalse value for its predicate or returns `nil` if no element matched:

```
(some even? [1 2 3])
-> true
```

```
(some even? [1 3 5])
-> nil
```

Notice that `some` does not end with a question mark. `some` is not a predicate, although it is often used like one. `some` returns the *actual value* of the first match instead of `true`. The distinction is invisible when you pair `some` with `even?`, since `even?` is itself a predicate. To see a non-true match, try using `some` with `identity` to find the first non-nil value in a sequence:

```
(some identity [nil false 1 nil 2])
-> 1
```

The behavior of the other predicates is obvious from their names:

```
(not-every? pred coll)
```

```
(not-any? pred coll)
```

Not every whole number is even:

```
(not-every? even? (whole-numbers))
-> true
```

But it would be a lie to claim that not any whole number is even:

```
(not-any? even? (whole-numbers))
-> false
```

Note that we picked questions to which we already knew the answer. In general, you have to be careful when applying predicates to infinite collections. They might run forever.

Transforming Sequences

Transformation functions transform the values in the sequence. The simplest transformation is `map`:

```
(map f coll)
```

`map` takes a source collection `coll` and a function `f`, and it returns a new sequence by invoking `f` on each element in the `coll`. You could use `map` to wrap every element in a collection with an HTML tag.

```
(map #(format "<p>%s</p>" %) ["the" "quick" "brown" "fox"])
-> ("<p>the</p>" "<p>quick</p>" "<p>brown</p>" "<p>fox</p>")
```

`map` can also take more than one collection argument. `f` must then be a function of multiple arguments. `map` will call `f` with one argument from each collection, stopping whenever the smallest collection is exhausted:

```
(map #(format "<%s>%s</%s>" %1 %2 %1)
["h1" "h2" "h3" "h1"] ["the" "quick" "brown" "fox"])
-> ("<h1>the</h1>" "<h2>quick</h2>" "<h3>brown</h3>"
"<h1>fox</h1>")
```

Another common transformation is reduce:

```
(reduce f coll)
```

`f` is a function of two arguments. `reduce` applies `f` on the first two elements in `coll` and then applies `f` to the result and the third element, and so on. `reduce` is useful for functions that “total up” a sequence in some way. You can use `reduce` to add items:

```
(reduce + (range 1 11))
-> 55
```

or to multiply them:

```
(reduce * (range 1 11))
-> 3628800
```

You can sort a collection with `sort` or `sort-by`:

```
(sort comp? coll)
```

```
(sort-by a-fn comp? coll)
```

`sort` sorts a collection by the natural order of its elements, where `sort-by` sorts a sequence by the result of calling `a-fn` on each element:

```
(sort [42 1 7 11])
-> (1 7 11 42)
```

```
(sort-by #(.toString %) [42 1 7 11])
-> (1 11 42 7)
```

If you do not want to sort by natural order, you can specify an optional comparison function `comp` for either `sort` or `sort-by`:

```
(sort > [42 1 7 11])
-> (42 11 7 1)
```

```
(sort-by :grade > [{:grade 83} {:grade 90} {:grade 77}])
-> ({:grade 90} {:grade 83} {:grade 77})
```

The granddaddy of all filters and transformations is the *list comprehension*. A list comprehension creates a list based on an existing list, using set notation. In other words, a comprehension states the properties that the result list must satisfy. In general, a list comprehension will consist of the following:

- Input list(s)
- Placeholder variables² for elements in the input lists

2. “Variables” in the mathematical sense, not the imperative programming sense. You can’t vary them. I humbly apologize for this overloading of the English language.

- Predicates on the elements
- An output form that produces output from the elements of the input lists that satisfy the predicates

Of course, Clojure generalizes the notion of list comprehension to *sequence* comprehension. Clojure comprehensions use the `for` macro.³

```
(for [binding-form coll-expr filter-expr? ...] expr)
```

`for` takes a vector of binding-form/coll-exprs, plus an optional filter-expr, and then yields a sequence of exprs.

List comprehension is more general than functions such as `map` and `filter` and can in fact emulate most of the filtering and transformation functions described earlier.

You can rewrite the previous `map` example as a list comprehension:

```
(for [word ["the" "quick" "brown" "fox"]]
  (format "<p>%s</p>" word))
-> ("<p>the</p>" "<p>quick</p>" "<p>brown</p>" "<p>fox</p>")
```

This reads almost like English: “For [each] word in [a sequence of words] format [according to format instructions].”

Comprehensions can emulate `filter` using a `:when` clause. You can pass `even?` to `:when` to filter the even numbers:

```
(take 10 (for [n (whole-numbers) :when (even? n)] n))
-> (2 4 6 8 10 12 14 16 18 20)
```

A `:while` clause continues the evaluation only while its expression holds true:

```
(for [n (whole-numbers) :while (even? n)] n)
-> ()
```

The real power of `for` comes when you work with more than one binding expression. For example, you can express all possible positions on a chess-board in algebraic notation by binding both rank and file:

```
(for [file "ABCDEFGH" rank (range 1 9)] (format "%c%d" file rank))
-> ("A1" "A2" ... elided ... "H7" "H8")
```

Clojure iterates over the rightmost binding expression in a sequence comprehension first and then works its way left. Because `rank` is listed to the right of `file` in the binding form, `rank` iterates faster. If you want files to iterate faster, you can reverse the binding order and list `rank` first:

3. The list comprehension `for` has nothing to do with the `for` loop found in imperative languages.

```
(for [rank (range 1 9) file "ABCDEFGH"] (format "%c%d" file rank))
-> ("A1" "B1" ... elided ... "G8" "H8")
```

In many languages, transformations, filters, and comprehensions do their work immediately. Do not assume this in Clojure. Most sequence functions do not traverse elements until you actually try to use them.

3.3 Lazy and Infinite Sequences

Most Clojure sequences are *lazy*; in other words, elements are not calculated until they are needed. Using lazy sequences has many benefits:

- You can postpone expensive computations that may not in fact be needed.
- You can work with huge data sets that do not fit into memory.
- You can delay I/O until it is absolutely needed.

Consider the code and following expression:

```
src/examples/primes.clj
(ns examples.primes)
;; Taken from clojure.contrib.lazy-seqs
;; primes cannot be written efficiently as a function, because
;; it needs to look back on the whole sequence. contrast with
;; fibs and powers-of-2 which only need a fixed buffer of 1 or 2
;; previous values.
(def primes
  (concat
    [2 3 5 7]
    (lazy-seq
      (let [primes-from
            (fn primes-from [n [f & r]]
              (if (some #(zero? (rem n %))
                      (take-while #(<= (* % %) n) primes))
                (recur (+ n f) r)
                (lazy-seq (cons n (primes-from (+ n f) r)))))]
        wheel (cycle [2 4 2 4 6 2 6 4 2 4 6 6 2 6 4 2
                      6 4 6 8 4 2 4 2 4 8 6 4 6 2 4 6
                      2 6 6 4 2 4 6 2 6 4 2 4 2 10 2 10]))]
      (primes-from 11 wheel))))))

(use 'examples.primes)
(def ordinals-and-primes (map vector (iterate inc 1) primes))
-> #'user/ordinals-and-primes
```

ordinals-and-primes includes pairs like [5, 11] (eleven is the fifth prime number). Both ordinals and primes are infinite, but ordinals-and-primes fits into memory just fine, because it is lazy. Just take what you need from it:

```
(take 5 (drop 1000 ordinals-and-primes))
-> ([1001 7927] [1002 7933] [1003 7937] [1004 7949] [1005 7951])
```

When should you prefer lazy sequences? Most of the time. Most sequence functions return lazy sequences, so you pay only for what you use. More important, lazy sequences do not require any special effort on your part. In the previous example, `iterate`, `primes`, and `map` return lazy sequences, so `ordinals-and-primes` gets laziness “for free.”

Lazy sequences are critical to functional programming in Clojure. [Section 4.2, *How to Be Lazy*, on page 90](#) explores creating and using lazy sequences in much greater detail.

Forcing Sequences

When you are viewing a large sequence from the REPL, you may want to use `take` to prevent the REPL from evaluating the entire sequence. In other contexts, you may have the opposite problem. You have created a lazy sequence, and you want to force the sequence to evaluate fully. The problem usually arises when the code generating the sequence has side effects. Consider the following sequence, which embeds side effects via `println`:

```
(def x (for [i (range 1 3)] (do (println i) i)))
-> #'user/x
```

Newcomers to Clojure are surprised that the previous code prints nothing. Since the definition of `x` does not actually use the elements, Clojure does not evaluate the comprehension to get them. You can force evaluation with `doall`:

```
(doall coll)
```

`doall` forces Clojure to walk the elements of a sequence and returns the elements as a result:

```
(doall x)
| 1
| 2
-> (1 2)
```

You can also use `dorun`:

```
(dorun coll)
```

`dorun` walks the elements of a sequence without keeping past elements in memory. As a result, `dorun` can walk collections too large to fit in memory.

```
(def x (for [i (range 1 3)] (do (println i) i)))
-> #'user/x
```

```
(dorun x)
| 1
| 2
-> nil
```

The nil return value is a telltale reminder that dorun does not hold a reference to the entire sequence. The dorun and doall functions help you deal with side effects, while most of the rest of Clojure discourages side effects. You should use these functions rarely.

3.4 Clojure Makes Java Seq-able

The seq abstraction of first/rest applies to anything that there can be more than one of. In the Java world, that includes the following:

- The Collections API
- Regular expressions
- File system traversal
- XML processing
- Relational database results

Clojure wraps these Java APIs, making the sequence library available for almost everything you do.

Seq-ing Java Collections

If you try to apply the sequence functions to Java collections, you will find that they behave as sequences. Collections that can act as sequences are called *seq-able*. For example, arrays are seq-able:

```
; String.getBytes returns a byte array
(first (.getBytes "hello"))
-> 104
```

```
(rest (.getBytes "hello"))
-> (101 108 108 111)
```

```
(cons (int \h) (.getBytes "ello"))
-> (104 101 108 108 111)
```

Hashtables and Maps are also seq-able:

```
; System.getProperties returns a Hashtable
(first (System/getProperties))
-> #<Entry java.runtime.name=Java(TM) SE Runtime Environment>
```

```
(rest (System/getProperties))
-> (#<Entry sun.boot.library.path=/System/Library/... etc. ...
```

Remember that the sequence wrappers are immutable, even when the underlying Java collection is mutable. So, you cannot update the system properties by consing a new item onto (System/getProperties). cons will return a new sequence; the existing properties are unchanged.

Since strings are sequences of characters, they also are seq-able:

```
(first "Hello")
-> \H

(rest "Hello")
-> (\e \l \l \o)

(cons \H "ello")
-> (\H \e \l \l \o)
```

Clojure will automatically wrap collections in sequences, but it will not automatically rewrap them back to their original type. With most collection types this behavior is intuitive, but with strings you will often want to convert the result to a string. Consider reversing a string. Clojure provides reverse:

```
; probably not what you want
(reverse "hello")
-> (\o \l \l \e \h)
```

To convert a sequence back to a string, use (apply str seq):

```
(apply str (reverse "hello"))
-> "olleh"
```

The Java collections are seq-able, but for most scenarios they do not offer advantages over Clojure's built-in collections. Prefer the Java collections only in interop scenarios where you are working with legacy Java APIs.

Seq-ing Regular Expressions

Clojure's regular expressions use the java.util.regex library under the hood. At the lowest level, this exposes the mutable nature of Java's Matcher. You can use re-matcher to create a Matcher for a regular expression and a string and then loop on re-find to iterate over the matches.

```
(re-matcher regexp string)

src/examples/sequences.clj
; don't do this!
(let [m (re-matcher #"\\w+" "the quick brown fox")]
  (loop [match (re-find m)]
    (when match
      (println match)
      (recur (re-find m)))))
```

```
| the
| quick
| brown
| fox
-> nil
```

Much better is to use the higher-level `re-seq`.

```
(re-seq regexp string)
```

`re-seq` exposes an immutable `seq` over the matches. This gives you the power of all of Clojure's sequence functions. Try these expressions at the REPL:

```
(re-seq #"\\w+" "the quick brown fox")
-> ("the" "quick" "brown" "fox")

(sort (re-seq #"\\w+" "the quick brown fox"))
-> ("brown" "fox" "quick" "the")

(drop 2 (re-seq #"\\w+" "the quick brown fox"))
-> ("brown" "fox")

(map #(.toUpperCase %) (re-seq #"\\w+" "the quick brown fox"))
-> ("THE" "QUICK" "BROWN" "FOX")
```

`re-seq` is a great example of how good abstractions reduce code bloat. Regular expression matches are not a special kind of thing, requiring special methods to deal with them. They are sequences, just like everything else. Thanks to the large number of sequence functions, you get more functionality *for free* than you would likely end up with after a misguided foray into writing regexp-specific functions.

Seq-ing the File System

You can `seq` over the file system. For starters, you can call `java.io.File` directly:

```
(import '(java.io File))
(.listFiles (File. "."))
-> [Ljava.io.File;@1f70f15e
```

The `[Ljava.io.File...` is Java's `toString()` representation for an array of `Files`. Sequence functions would call `seq` on this automatically, but the REPL doesn't.

So, `seq` it yourself:

```
(seq (.listFiles (File. ".")) )
-> (#<./concurrency> #<./sequences> ...)
```

If the default print format for files does not suit you, you could map them to a string form with `getName`:

```
; overkill
(map #(.getName %) (seq (.listFiles (File. "."))))
-> ("concurrency" "sequences" ...)
```

Once you decide to use a function like `map`, calling `seq` is redundant. Sequence library functions call `seq` for you, so you don't have to. The previous code simplifies to this:

```
(map #(.getName %) (.listFiles (File. ".")))
-> ("concurrency" "sequences" ...)
```

Often, you want to recursively traverse the entire directory tree. Clojure provides a depth-first walk via `file-seq`. If you file-seq from the sample code directory for this book, you will see a lot of files:

```
(count (file-seq (File. ".")))
-> 104 ; the final number will be larger!
```

What if you want to see only the files that have been changed recently? Write a predicate `recently-modified?` that checks to see whether File was touched in the last half hour:

```
src/examples/sequences.clj
(defn minutes-to-millis [mins] (* mins 1000 60))

(defn recently-modified? [file]
  (> (.lastModified file)
    (- (System/currentTimeMillis) (minutes-to-millis 30))))
```

Give it a try: ⁴

```
(filter recently-modified? (file-seq (File. ".")))
-> (./sequences ./sequences/sequences.clj)
```

Seq-ing a Stream

You can seq over the lines of any Java Reader using `line-seq`. To get a Reader, you can use Clojure's `clojure.java.io` library. The `clojure.java.io` library provides a reader function that returns a reader on a stream, file, URL, or URI.

```
(use '[clojure.java.io :only (reader)])
; leaves reader open...
(take 2 (line-seq (reader "src/examples/utls.clj")))
-> ("(ns examples.utls" " (:import [java.io BufferedReader InputStreamReader]))")
```

Since readers can represent nonmemory resources that need to be closed, you should wrap reader creation in a `with-open`. Create an expression that uses

4. Your results will vary from those shown here.

the sequence function `count` to count the number of lines in a file and uses `with-open` to correctly close the reader:

```
(with-open [rdr (reader "src/examples/utils.clj")]
  (count (line-seq rdr)))
-> 64
```

To make the example more useful, add a filter to count only nonblank lines:

```
(with-open [rdr (reader "src/examples/utils.clj")]
  (count (filter #(re-find #"\S" %) (line-seq rdr))))
-> 55
```

Using seqs both on the file system and on the contents of individual files, you can quickly create interesting utilities. Create a program that defines these three predicates:

- `non-blank?` detects nonblank lines.
- `non-svn?` detects files that are not Subversion metadata.
- `clojure-source?` detects Clojure source code files.

Then, create a `clojure-loc` function that counts the lines of Clojure code in a directory tree, using a combination of sequence functions along the way: `reduce`, `for`, `count`, and `filter`.

```
src/examples/sequences.clj
(use '[clojure.java.io :only (reader)])
(defn non-blank? [line] (if (re-find #"\S" line) true false))

(defn non-svn? [file] (not (.contains (.toString file) ".svn")))

(defn clojure-source? [file] (.endsWith (.toString file) ".clj"))

(defn clojure-loc [base-file]
  (reduce
    +
    (for [file (file-seq base-file)
          :when (and (clojure-source? file) (non-svn? file))]
      (with-open [rdr (reader file)]
        (count (filter non-blank? (line-seq rdr)))))))
```

Now let's use `clojure-loc` to find out how much Clojure code is in Clojure itself:

```
(clojure-loc (java.io.File. "/home/abedra/src/opensource/clojure/clojure"))
-> 38716
```

The `clojure-loc` function is very task-specific, but because it is built out of sequence functions and simple predicates, you can easily tweak it to very different tasks.

Seq-ing XML

Clojure can seq over XML data. The examples that follow use this XML:

```
data/sequences/compositions.xml
<compositions>
  <composition composer="J. S. Bach">
    <name>The Art of the Fugue</name>
  </composition>
  <composition composer="F. Chopin">
    <name>Fantaisie-Improptu Op. 66</name>
  </composition>
  <composition composer="W. A. Mozart">
    <name>Requiem</name>
  </composition>
</compositions>
```

The function `clojure.xml/parse` parses an XML file/stream/URI, returning the tree of data as a Clojure map, with nested vectors for descendants:

```
(use '[clojure.xml :only (parse)])
(parse (java.io.File. "data/sequences/compositions.xml"))
-> {:tag :compositions,
   :attrs nil,
   :content [{:tag :composition, ... etc. ...}]}
```

You can manipulate this map directly, or you can use the `xml-seq` function to view the tree as a seq:

```
(xml-seq root)
```

The following example uses a list comprehension over an `xml-seq` to extract just the composers:

```
src/examples/sequences.clj
(for [x (xml-seq
  (parse (java.io.File. "data/sequences/compositions.xml")))
  :when (= :composition (:tag x))]
  (:composer (:attrs x)))
-> ("J. S. Bach" "F. Chopin" "W. A. Mozart")
```

3.5 Calling Structure-Specific Functions

Clojure's sequence functions allow you to write very general code. Sometimes you will want to be more specific and take advantage of the characteristics of a specific data structure. Clojure includes functions that specifically target lists, vectors, maps, structs, and sets.

We will take a quick tour of some of these structure-specific functions next. For a complete list of structure-specific functions in Clojure, see the Data Structures section of the Clojure website.⁵

Functions on Lists

Clojure supports the traditional names `peek` and `pop` for retrieving the first element of a list and the remainder, respectively:

```
(peek coll)
```

```
(pop coll)
```

Give a simple list a peek and pop:

```
(peek '(1 2 3))
-> 1
```

```
(pop '(1 2 3))
-> (2 3)
```

`peek` is the same as `first`, but `pop` is *not* the same as `rest`. `pop` will throw an exception if the sequence is empty:

```
(rest ())
-> ()
```

```
(pop ())
-> java.lang.IllegalStateException: Can't pop empty list
```

Functions on Vectors

Vectors also support `peek` and `pop`, but they deal with the element at the end of the vector:

```
(peek [1 2 3])
-> 3
```

```
(pop [1 2 3])
-> [1 2]
```

`get` returns the value at an index or returns `nil` if the index is outside the vector:

```
(get [:a :b :c] 1)
-> :b
```

```
(get [:a :b :c] 5)
-> nil
```

5. http://clojure.org/data_structures

Vectors are themselves functions. They take an index argument and return a value, or they throw an exception if the index is out of bounds:

```
([:a :b :c] 1)
-> :b
```

```
([:a :b :c] 5)
-> java.lang.ArrayIndexOutOfBoundsException: 5
```

`assoc` associates a new value with a particular index:

```
(assoc [0 1 2 3 4] 2 :two)
-> [0 1 :two 3 4]
```

`subvec` returns a subvector of a vector:

```
(subvec avec start end?)
```

If `end` is not specified, it defaults to the end of the vector:

```
(subvec [1 2 3 4 5] 3)
-> [4 5]
```

```
(subvec [1 2 3 4 5] 1 3)
-> [2 3]
```

Of course, you could simulate `subvec` with a combination of `drop` and `take`:

```
(take 2 (drop 1 [1 2 3 4 5]))
-> (2 3)
```

The difference is that `take` and `drop` are general and can work with any sequence. On the other hand, `subvec` is *much* faster for vectors. Whenever a structure-specific function like `subvec` duplicates functionality already available in the sequence library, it is probably there for performance. The documentation string for functions like `subvec` includes performance characteristics.

Functions on Maps

Clojure provides several functions for reading the keys and values in a map. `keys` returns a sequence of the keys, and `vals` returns a sequence of the values:

```
(keys map)
```

```
(vals map)
```

Try taking keys and values from a simple map:

```
(keys {:sundance "spaniel", :darwin "beagle"})
-> (:sundance :darwin)
```

```
(vals {:sundance "spaniel", :darwin "beagle"})
-> ("spaniel" "beagle")
```

get returns the value for a key or returns nil.

```
(get map key value-if-not-found?)
```

Use your REPL to test that get behaves as expected for keys both present and missing:

```
(get {:sundance "spaniel", :darwin "beagle"} :darwin)
-> "beagle"
```

```
(get {:sundance "spaniel", :darwin "beagle"} :snoopy)
-> nil
```

There is an approach even simpler than get. Maps are functions of their keys. So, you can leave out the get entirely, putting the map in function position at the beginning of a form:

```
({ :sundance "spaniel", :darwin "beagle" } :darwin)
-> "beagle"
```

```
({ :sundance "spaniel", :darwin "beagle" } :snoopy)
-> nil
```

Keywords are also functions. They take a collection as an argument and look themselves up in the collection. Since :darwin and :sundance are keywords, the earlier forms can be written with their elements in reverse order.

```
(:darwin { :sundance "spaniel", :darwin "beagle" } )
-> "beagle"
```

```
(:snoopy { :sundance "spaniel", :darwin "beagle" } )
-> nil
```

If you look up a key in a map and get nil back, you cannot tell whether the key was missing from the map or present with a value of nil. The contains? function solves this problem by testing for the mere presence of a key.

```
(contains? map key)
```

Create a map where nil is a legal value:

```
(def score {:stu nil :joey 100})
```

:stu is present, but if you see the nil value, you might not think so:

```
(:stu score)
-> nil
```

If you use `contains?`, you can verify that `:stu` is in the game, although presumably not doing very well:

```
(contains? score :stu)
-> true
```

Another approach is to call `get`, passing in an optional third argument that will be returned if the key is not found:

```
(get score :stu :score-not-found)
-> nil

(get score :aaron :score-not-found)
-> :score-not-found
```

The default return value of `:score-not-found` makes it possible to distinguish that `:aaron` is not in the map, while `:stu` is present with a value of `nil`.

If `nil` is a legal value in map, use `contains?` or the three-argument form of `get` to test the presence of a key.

Clojure also provides several functions for building new maps:

- `assoc` returns a map with a key/value pair added.
- `dissoc` returns a map with a key removed.
- `select-keys` returns a map, keeping only the keys passed in.
- `merge` combines maps. If multiple maps contain a key, the rightmost map wins.

To test these functions, create some song data:

```
src/examples/sequences.clj
(def song {:name "Agnus Dei"
           :artist "Krzysztof Penderecki"
           :album "Polish Requiem"
           :genre "Classical"})
```

Next, create various modified versions of the song collection:

```
(assoc song :kind "MPEG Audio File")
-> {:name "Agnus Dei", :album "Polish Requiem",
   :kind "MPEG Audio File", :genre "Classical",
   :artist "Krzysztof Penderecki"}

(dissoc song :genre)
-> {:name "Agnus Dei", :album "Polish Requiem",
   :artist "Krzysztof Penderecki"}

(select-keys song [:name :artist])
-> {:name "Agnus Dei", :artist "Krzysztof Penderecki"}
```

```
(merge song {:size 8118166, :time 507245})
-> {:name "Agnus Dei", :album "Polish Requiem",
:genre "Classical", :size 8118166,
:artist "Krzysztof Penderecki", :time 507245}
```

Remember that song itself never changes. Each of the functions shown previously returns a new collection.

The most interesting map construction function is merge-with.

```
(merge-with merge-fn & maps)
```

merge-with is like merge, except that when two or more maps have the same key, you can specify your own function for combining the values under the key. Use merge-with and concat to build a sequence of values under each key:

```
(merge-with
 concat
 {:rubble ["Barney"], :flintstone ["Fred"]}
 {:rubble ["Betty"], :flintstone ["Wilma"]}
 {:rubble ["Bam-Bam"], :flintstone ["Pebbles"]})
-> {:rubble ("Barney" "Betty" "Bam-Bam"),
:flintstone ("Fred" "Wilma" "Pebbles")}
```

Starting with three distinct collections of family members keyed by last name, the previous code combines them into one collection keyed by last name.

Functions on Sets

In addition to the set functions in the clojure namespace, Clojure provides a group of functions in the clojure.set namespace. To use these functions with unqualified names, call (use 'clojure.set) from the REPL. For the following examples, you will also need the following vars:

```
src/examples/sequences.clj
(def languages #{"java" "c" "d" "clojure"})
(def beverages #{"java" "chai" "pop"})
```

The first group of clojure.set functions performs operations from set theory:

- union returns the set of all elements present in either input set.
- intersection returns the set of all elements present in *both* input sets.
- difference returns the set of all elements present in the first input set, minus those in the second.
- select returns the set of all elements matching a predicate.

Write an expression that finds the union of all languages and beverages:

```
(union languages beverages)
-> #{"java" "c" "d" "clojure" "chai" "pop"}
```

Next, try the languages that are not also beverages:

```
(difference languages beverages)
-> #{ "c" "d" "clojure" }
```

If you enjoy terrible puns, you will like the fact that some things are both languages *and* beverages:

```
(intersection languages beverages)
-> #{ "java" }
```

A surprising number of languages cannot afford a name larger than a single character:

```
(select #(= 1 (.length %)) languages)
-> #{ "c" "d" }
```

Set union and difference are part of set theory, but they are also part of *relational algebra*, which is the basis for query languages such as SQL. The relational algebra consists of six primitive operators: set union and set difference (described earlier), plus rename, selection, projection, and cross product.

You can understand the relational primitives by following the analogy with relational databases (see the following table).

Relational Algebra	Database	Clojure Type System
Relation	Table	Anything set-like
Tuple	Row	Anything map-like

The following examples work against an in-memory database of musical compositions. Load the database before continuing:

```
src/examples/sequences.clj
(def compositions
  #{{:name "The Art of the Fugue" :composer "J. S. Bach"}
    {:name "Musical Offering" :composer "J. S. Bach"}
    {:name "Requiem" :composer "Giuseppe Verdi"}
    {:name "Requiem" :composer "W. A. Mozart"}})
(def composers
  #{{:composer "J. S. Bach" :country "Germany"}
    {:composer "W. A. Mozart" :country "Austria"}
    {:composer "Giuseppe Verdi" :country "Italy"}})
(def nations
  #{{:nation "Germany" :language "German"}
    {:nation "Austria" :language "German"}
    {:nation "Italy" :language "Italian"}})
```

The rename function renames keys (*database columns*), based on a map from original names to new names.

```
(rename relation rename-map)
```

Rename the compositions to use a title key instead of name:

```
(rename compositions {:name :title})
-> #{{:title "Requiem", :composer "Giuseppe Verdi"}
{:title "Musical Offering", :composer "J.S. Bach"}
{:title "Requiem", :composer "W. A. Mozart"}
{:title "The Art of the Fugue", :composer "J.S. Bach"}}
```

The select function returns maps for which a predicate is true and is analogous to the WHERE portion of a SQL SELECT:

```
(select pred relation)
```

Write a select expression that finds all the compositions whose title is "Requiem":

```
(select #(= (:name %) "Requiem") compositions)
-> #{{:name "Requiem", :composer "W. A. Mozart"}
{:name "Requiem", :composer "Giuseppe Verdi"}}
```

The project function returns only the parts of maps that match a set of keys.

```
(project relation keys)
```

project is similar to a SQL SELECT that specifies a subset of columns. Write a projection that returns only the name of the compositions:

```
(project compositions [:name])
-> #{{:name "Musical Offering"}
{:name "Requiem"}
{:name "The Art of the Fugue"}}
```

The final relational primitive, which is a cross product, is the foundation for the various kinds of joins in relational databases. The cross product returns every possible combination of rows in the different tables. You can do this easily enough in Clojure with a list comprehension:

```
(for [m compositions c composers] (concat m c))
-> ... 4 x 3 = 12 rows ...
```

Although the cross product is theoretically interesting, you will typically want some subset of the full cross product. For example, you might want to join sets based on shared keys:

```
(join relation-1 relation-2 keymap?)
```

You can join the composition names and composers on the shared key :composer:

```
(join compositions composers)
-> #{{:name "Requiem", :country "Austria",
:composer "W. A. Mozart"}}
```

```
{:name "Musical Offering", :country "Germany",
:composer "J. S. Bach"}
{:name "Requiem", :country "Italy",
:composer "Giuseppe Verdi"}
{:name "The Art of the Fugue", :country "Germany",
:composer "J. S. Bach"}}
```

If the key names in the two relations do not match, you can pass a keymap that maps the key names in relation-1 to their corresponding keys in relation-2. For example, you can join composers, which uses `:country`, to nations, which uses `:nation`. For example:

```
(join composers nations {:country :nation})
-> #{{:language "German", :nation "Austria",
:composer "W. A. Mozart", :country "Austria"}
{:language "German", :nation "Germany",
:composer "J. S. Bach", :country "Germany"}
{:language "Italian", :nation "Italy",
:composer "Giuseppe Verdi", :country "Italy"}}
```

You can combine the relational primitives. Perhaps you want to know the set of all countries that are home to the composer of a requiem. You can use `select` to find all the requiems, join them with their composers, and project to narrow the results to just the country names:

```
(project
(join
(select #(= (:name %)) "Requiem") compositions)
composers)
[:country])
-> #{{:country "Italy"} {:country "Austria"}}
```

The analogy between Clojure's relational algebra and a relational database is instructive. Remember, though, that Clojure's relational algebra is a general-purpose tool. You can use it on any kind of set-relational data. And while you're using it, you have the entire power of Clojure and Java at your disposal.

3.6 Wrapping Up

Clojure unifies all kinds of collections under a single abstraction, the sequence. After more than a decade dominated by object-oriented programming, Clojure's sequence library is the "Revenge of the Verbs."

Clojure's sequences are implemented using functional programming techniques: immutable data, recursive definition, and lazy realization. In the next chapter, you will see how to use these techniques directly, further expanding the power of Clojure.